
OPENCV DESDE PYTHON:

INTRODUCCIÓN AL PROCESAMIENTO DE IMÁGENES

INTRODUCCIÓN

En los dos proyectos a desarrollar en esta asignatura, se deberá resolver de manera óptima la detección y/o el seguimiento de al menos un objeto de interés real. Para resolver esta tarea, se deberán analizar todos los métodos conocidos y seleccionar el óptimo según requiera nuestra aplicación.

OBJETIVOS

El objetivo principal de esta sesión de prácticas es comparar algunos métodos de detección y seguimiento de un objeto de interés en una secuencia de imágenes desde Python utilizando OpenCV 4.5.

MATERIALES

En el caso que no hayas cursado la asignatura de “Visión por Computador” te recomiendo analizar el siguiente material:

- PoliformaT -> Recursos/2-Software-&-cuadernos/30-Jupyter lab Tutorial - Image preprocessing
- PoliformaT -> Recursos/2-Software-&-cuadernos/31-Jupyter lab Tutorial - Image segmentation

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/32-PyCharm demo - Image processing
- PoliformaT -> Recursos/2-Software-&-cuadernos/33-Jupyter lab Tutorial - ArUco detection
- PoliformaT -> Recursos/2-Software-&-cuadernos/34-Jupyter lab Tutorial - Trackers

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

```
(base) C:\windows\system32> conda activate open3d_env  
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working_folder_path”
```

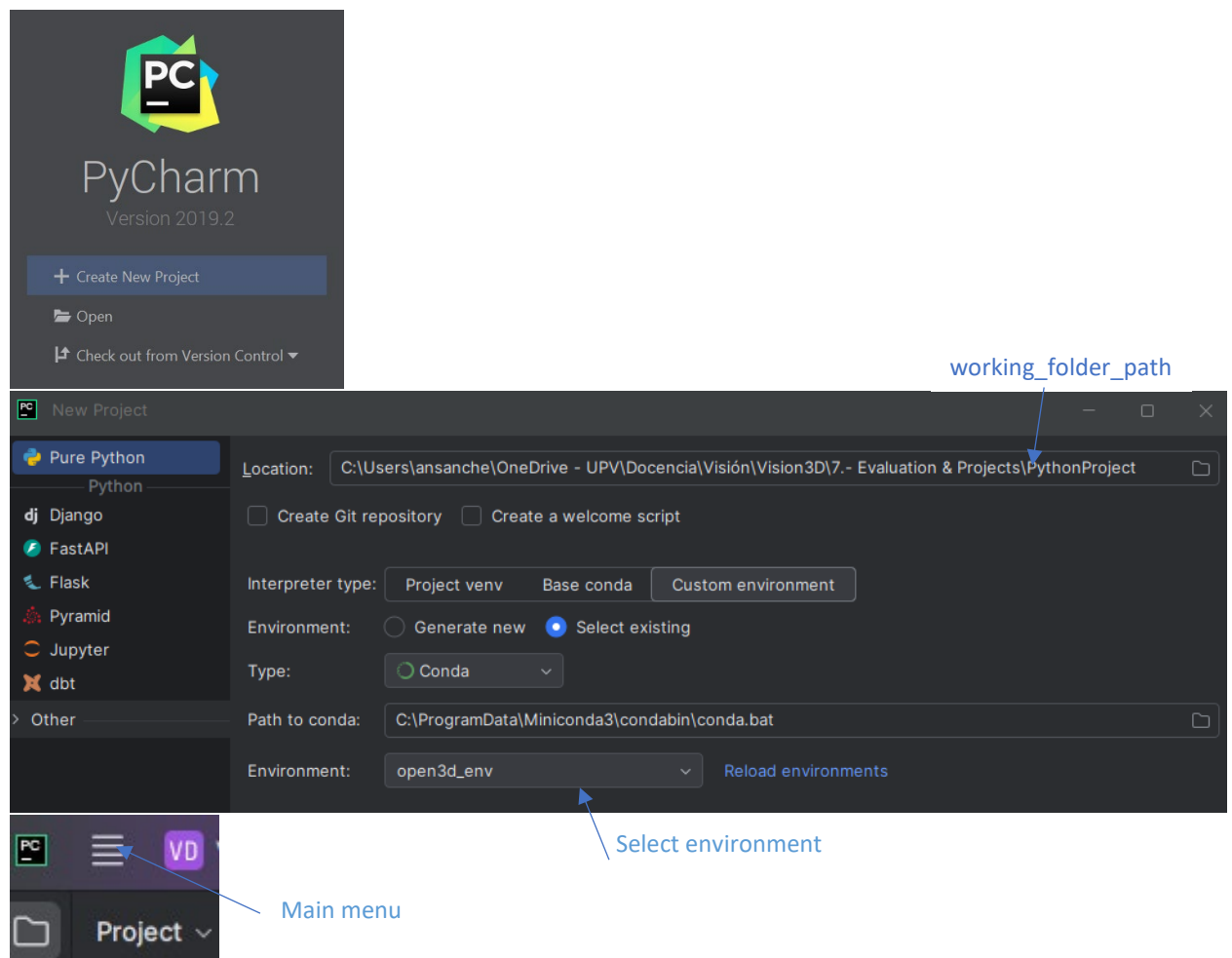
Después de arrancar Jupyter lab, escribe en una célula de código el comando: `"import cv2"`

Mediante este módulo, Python encontrará todos los métodos de Python que sirven de interfaz entre la librería OpenCV de C++ y Python.

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

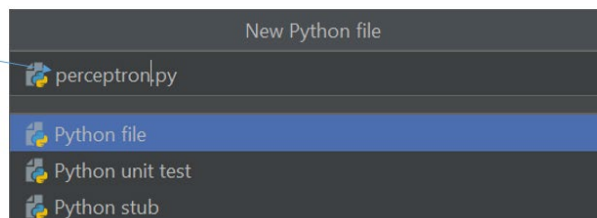
Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



1.- DETECCIÓN DE OBJETOS DE INTERÉS

En el cuaderno '32_Image_processing.ipynb', analizar la celda "*Contours demo*" con la imagen fishBinary.bmp. En esta celda se obtienen los contornos de los objetos de interés de una imagen binaria (bw). Para etiquetarlos y obtener las máscaras de cada uno de ellos se puede usar:

- `contours, hierarchy = cv2.findContours(bw, cv2.RETR_CCOMP, ...
cv2.CHAIN_APPROX_SIMPLE)` (más rápido)
- `num, labels, stats, centroids =
cv2.connectedComponentsWithStats(bw)`
(más fácil de usar)

En el caso de utilizar `cv2.findContours`, para obtener los centroides los podemos calcular de esta forma:

```
- m = cv2.moments(contorno1)
```

El centroide (x, y) se calcula como: $(m.m10/m.m00, m.m01/m.m00)$

```
- area = cv2.contourArea(contorno1)
```

Si utilizamos `cv2.connectedComponents` ya nos devuelve los centroides como uno de los argumentos de salida.

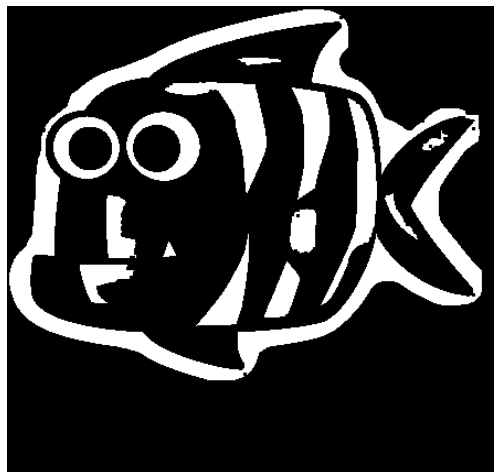


ILUSTRACIÓN 1: IMAGEN FISHBINARY.BMP

2.- SEGMENTACIÓN POR DIFERENCIAS

Para la segmentación utilizaremos el método **createBackgroundSubtractorMOG2**: "*Gaussian Mixture-based Background/Foreground Segmentation Algorithm*" [Zivkovic2006]. Como indica su nombre este método calcula la máscara (nuestro objeto de interés) realizando una sustracción de la imagen con el objeto y una imagen del fondo (background). En los videos el background se tiene que ir actualizando en el caso de que el fondo no sea estático. Otra opción muy eficiente cuando los objetos de interés son pequeños es utilizar el método **createBackgroundSubtractorKNN** que funciona de forma muy similar.

En la celda "*Segmentation demo*" del cuaderno '32_Image_processing.ipynb' se encuentra una demo de su utilización en un vídeo.

Las principales partes de dicha celda son:

- El constructor del objeto. VarThreshold indica un umbral de varianza para determinar si queremos más o menos foreground detectado.

```
#Create a background subtractor
backSub = cv2.createBackgroundSubtractorMOG2 (None,
VarThreshold)
```

- Utilizar el objeto. Esta función se le pasa la imagen (frame) y parámetro LearningRate que es un número que pondremos a 0 si no queremos actualizar el background o a -1 si queremos que el método decida si actualizarlo o no. Esta función nos devuelve la imagen binarizada fgmask donde tendremos nuestro objeto de segmentado.

```
fgmask = backSub.apply(frame, None, learnRate)
```

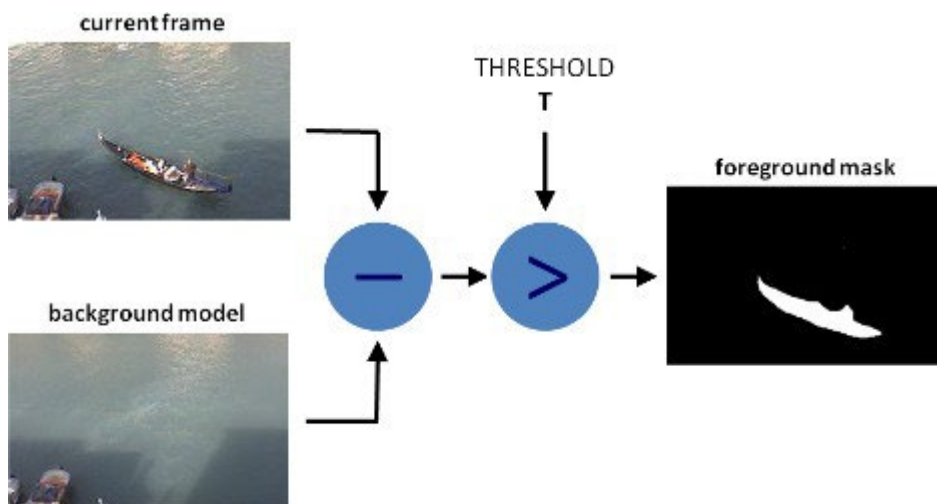


ILUSTRACIÓN 2: EJEMPLO DE SUBTRACCIÓN DE FONDO

REFERENCIAS

[Zivkovic2006]:

Zoran Zivkovic and Ferdinand van der Heijden. "Efficient adaptive density estimation per image pixel for the task of background subtraction". Pattern recognition letters, 27(7):773-780, 2006.

[PDF](<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=a42aff222c6c8bc248a0504d063ef7334fe5a177>).

Tutorial Background Subtraction Methods: https://docs.opencv.org/4.5.0/d1/dc5/tutorial_background_subtraction.html

3.-TAREA

Deberéis desarrollar un detector y un seguidor de al menos un objeto real en movimiento durante la secuencia de imágenes capturada por la cámara. La detección se realizará únicamente sobre la primera imagen de la secuencia. Mientras que el seguimiento se realizará en el resto de las imágenes de la secuencia, aprovechando la información de las detecciones previas. Por ejemplo, se puede detectar y seguir objetos reales en movimiento como una pelota, un marcador móvil o una persona. Por otra parte, también se puede utilizar marcadores fijos para definir la localización de algún objeto virtual (como un agujero virtual, una meta virtual, etc.). Para empezar, se recomienda resolver la tarea de detección de manera manual, pulsando con el ratón en el centro de cada objeto sobre las imágenes (o seleccionando la región de interés que los contiene). Después, durante el desarrollo del proyecto también se deberá automatizar el proceso de detección de los objetos utilizando técnicas automáticas desarrolladas con funcionalidad propia o de OpenCV y estimar el centroide del objeto durante toda la secuencia de imágenes.